ADR 0019 — One mutation at a time per shared repository

- **Status:** Accepted
- **Date:** 2026-07-25
- **Milestone / issue:** M1.07 — Serialize mutations per worktree (#60)
- **Supersedes / superseded by:** — (makes ADR 0018's staleness gate load-bearing under concurrency; ADR 0016's single funnel is the seam this attaches to)

Context

Since #143 every write takes one path, and since #145 that path re-verifies the plan against the live repository immediately before executing. That gate is a **detector**, not a lock. Nothing stopped two requests from running the pipeline at once:

t	Request A	Request B
1	`build_plan` — generation G	
2		`build_plan` — generation G
3	`enforce_fresh` — still G, passes	
4		`enforce_fresh` — still G, passes
5	`execute` — commits	
6		`execute` — commits **again**

A double-clicked Commit made two commits. Both requests were honest; the gate was asked the same question twice before either answer was acted on.

Second, independent problem: git's refs, packed-refs and object store are shared by **every linked worktree of one clone**. Two worktrees racing on `refs/heads/x` corrupt each other even though they are different working trees.

Decision

A new `coordinator` module owns one `tokio::sync::Mutex` per shared repository, keyed by `RepositoryId`. `planner::plan_and_execute_in` acquires it and holds it across `validate → enforce_fresh → execute`.

The plan is built before the guard is taken, deliberately. The obvious arrangement — guard first, then observe — serializes correctly and silently defeats the purpose: the queued duplicate would wait, observe the new state, build a perfectly fresh plan, and commit a second time. Both commits would be individually valid and nothing would ever look stale. Building first means the second request carries the pre-mutation generation into the guard, where `enforce_fresh` sees the drift and refuses it. The guard serializes; the #145 gate decides. This ordering is pinned by a source-level test

(`the_production_entry_point_composes_the_tested_stages_in_order`), because it is the kind of thing a later refactor "tidies" into a bug.

The key is `RepositoryId`, not `WorktreeId`. `RepositoryId` is derived from the shared common directory, so every linked worktree of one clone maps to one guard — exactly the set that shares a ref store. A per-worktree key would leave two linked worktrees free to race on the same ref.

Waiters are the queue. `tokio::sync::Mutex` is FIFO-fair, so there is no separate queue type. Cancellation before start therefore comes for free: axum drops the handler future when the client disconnects, dropping a pending acquire removes that waiter, and git never runs.

Degraded mode still serializes. A served path that will not classify as a repository has no id; those writes share one fallback guard rather than skipping serialization. "We don't know which repository this is" must never mean "so let them all run at once".

External git is detected, not excluded. Before planning, the coordinator stats `index.lock` in the worktree's real git directory (resolved with `git rev-parse --absolute-git-dir`, since a linked worktree's `.git` is a file and its index lives under the common dir). If present, the request is refused 409 with "Another git process is working in this repository — wait for it to finish and try again." This is a **courtesy check, not a guarantee** — the external process can take the lock a moment later, and when it does git refuses on its own and its stderr is forwarded verbatim, exactly as before. Git's own lock file remains the real mutual exclusion against processes outside this server.

Alternatives considered

- **An actor task per repository with an mpsc queue.** The issue's wording ("mutation coordinator", "actor restart behavior is documented") presupposed it. Rejected: it buys an introspectable queue and explicit cancellation, and costs a supervised task with a death-and-restart story, a cancellation flag per queued job, and every operation's data crossing a channel. At one user, none of that is earned — the lock gets FIFO ordering and cancellation for free. Recorded here so the roadmap's phrasing is not read as an unmet promise.
- **Refuse immediately with 409-busy instead of queueing.** Simplest and deadlock-proof, but every honest concurrent click becomes a user-visible error, and "queued operations can be cancelled before start" has nothing to cancel.
- **Keying by `WorktreeId`.** Matches the issue title literally and is wrong: linked worktrees share the ref store. The per-repository key is a superset and satisfies the title by construction.
- **Guarding the observation as well as the mutation.** See above — it is the arrangement that looks right and reintroduces the double-commit. A test caught it during implementation, not review.

Consequences

1. **There is no actor, so there is no restart behavior.** No long-lived owner task exists to die: no restart path, no message loss, no queue to drain. The failure mode a lock has

instead is a **panicking holder** — `tokio::sync::Mutex` does not poison, so unwinding releases the guard and the next waiter proceeds against a repository that may be half-mutated. That exposure is unchanged by this work (a panic mid-`execute` left the same state before the guard existed), the staleness gate re-checks on the next operation, and rollback of a half-executed plan is explicitly out of scope.
2. **Two genuinely different concurrent operations also end with one refusal**, since the loser's generation moved too. Accepted: at one user that is a retry, and the alternative is duplicate mutations. Revisit if the refusal ever becomes common in practice.
3. **`StageAll`/`UnstageAll` are serialized more than strictly necessary** — they touch only one worktree's index. This is the deliberate loosening point if per-worktree parallelism is ever wanted.
4. **The wait queue is unbounded** — no depth limit, no 429. At one user that is not a denial-of-service surface; a bound would invent a refusal nobody can trigger. Revisit if multi-client ever lands.
5. **`spawn_blocking` covers the planner path only.** `read_head_branch`, `read_refs`, and the journal writes on that path moved off the async worker threads; the read handlers were **not** swept. Do not read this ADR as "done everywhere".
6. **No protocol change.** Nothing in `git-vista-protocol` moved, so no version bump (ADR 0002). One new user-visible string, the `index.lock` 409.

Where this is implemented

- `crates/git-vista-server/src/coordinator.rs` — the guard registry and the external-git busy check.
- `crates/git-vista-server/src/planner.rs` — `plan_and_execute_in`, the guarded pipeline, plus the blocking-work offload helpers.
- `crates/git-vista-server/src/planner/coordination_suite.rs` — the acceptance tests: double-click, no-interleaving, cancellation, linked-worktree races, busy repository, reads-keep-running, and the source pin on blocking calls.
- `crates/git-vista-server/src/planner/contract_suite.rs` — the composition pin, now covering the guard's position.

Signed: thomas2010 · 2026-07-25T02:05:00-04:00